

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

QUESTIONS:

Total Marks: 30

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

- (a) Analyze the possible reasons for lack of responsiveness in the existing layout.
- (b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.
- (c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.
- (d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
- (e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.

- (a) Analyze the issues in the existing form validation mechanism.
- (b) Design input fields for email and phone number with appropriate HTML attributes.
- (c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.
- (d) Develop a mechanism to display **real-time error messages** for invalid inputs.
- (e) Evaluate the validation system and suggest enhancements for better user experience and security.

Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.

- Analyze the CSS properties responsible for layout overlapping issues.
- Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.
- Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.
- Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
- Evaluate the corrected layout and suggest best practices for maintaining consistency.

Code of Conduct:

I Ebin Antony P (192373011) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Responsive Layout Design

Aim: To design a responsive retail webpage using CSS Flexbox and Media Queries that works properly on mobile, tablet, and desktop screens.

(a) Possible reasons for lack of responsiveness

The existing website may not be responsive because of:

1. Fixed widths such as width: 1000px.
2. Excessive use of absolute positioning.
3. Missing viewport meta tag.
4. No CSS media queries.
5. Improper use of margins and padding.
6. Images having fixed dimensions.
7. Navigation items not wrapping on smaller screens.
8. Using pixels instead of flexible units such as %, rem, vw, and fr.

(b), (c), (d) Responsive webpage implementation

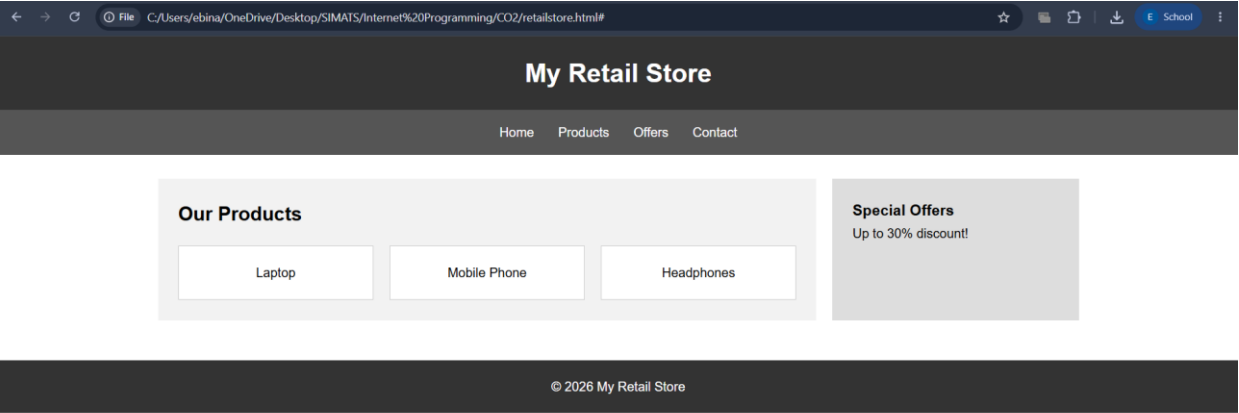
```
<html>
<head>
  <title>Responsive Retail Website</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <style>
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }
    body {
      font-family: Arial, sans-serif;
      line-height: 1.6;
    }
    header {
      background: #333;
      color: white;
      padding: 20px;
      text-align: center;
    }
    nav {
      background: #555;
      display: flex;
      justify-content: center;
      gap: 30px;
      padding: 15px;
    }
    nav a {
      color: white;
      text-decoration: none;
    }
    .container {
      display: flex;
      gap: 20px;
```

```

padding: 30px;
max-width: 1200px;
margin: auto;
}
.content {
  flex: 3;
  background: #f2f2f2;
  padding: 25px;
}
.sidebar {
  flex: 1;
  background: #ddd;
  padding: 25px;
}
.products {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
  margin-top: 20px;
}
.product {
  background: white;
  border: 1px solid #ccc;
  padding: 20px;
  text-align: center;
}
footer {
  background: #333;
  color: white;
  text-align: center;
  padding: 20px;
  margin-top: 20px;
}
/* Tablet */
@media (max-width: 768px) {
  .container {
    flex-direction: column;
    padding: 20px;
  }
  .products {
    grid-template-columns: repeat(2, 1fr);
  }
  nav {
    gap: 15px;
  }
}
/* Mobile */
@media (max-width: 480px) {
  nav {
    flex-direction: column;
    align-items: center;
    gap: 10px;
  }
  .container {
    padding: 10px;
  }
  .products {

```

```
        grid-template-columns: 1fr;
    }
    header {
        padding: 15px;
    }
}
</style>
</head>
<body>
<header>
    <h1>My Retail Store</h1>
</header>
<nav>
    <a href="#">Home</a>
    <a href="#">Products</a>
    <a href="#">Offers</a>
    <a href="#">Contact</a>
</nav>
<div class="container">
    <section class="content">
        <h2>Our Products</h2>
        <div class="products">
            <div class="product">Laptop</div>
            <div class="product">Mobile Phone</div>
            <div class="product">Headphones</div>
        </div>
    </section>
    <aside class="sidebar">
        <h3>Special Offers</h3>
        <p>Up to 30% discount!</p>
    </aside>
</div>
<footer>
    <p>© 2026 My Retail Store</p>
</footer>
</body>
</html>
```



(e) Evaluation

The responsive design is effective because:

- **Desktop:** Products are displayed in three columns.
- **Tablet:** Products are displayed in two columns.
- **Mobile:** Products are displayed in a single column.
- Navigation changes from horizontal to vertical on mobile.
- Flexbox maintains proper alignment.
- Grid provides consistent product spacing.
- box-sizing: border-box prevents unwanted size calculations.

Further improvements

- Use responsive images.
- Add accessible navigation.
- Optimize page loading speed.
- Use CSS variables for consistent colors and spacing.
- Test on more browsers and devices.

Experiment 2: Form Validation Using JavaScript

Aim: To create a registration form and validate email and phone number using HTML attributes, JavaScript, and Regular Expressions.

(a) Problems in existing validation

Possible problems include:

1. No validation of email format.
2. Phone numbers accepting letters.
3. Empty fields being submitted.
4. Validation occurring only after form submission.
5. No meaningful error messages.
6. Client-side validation being treated as the only security mechanism.

(b), (c), (d) Implementation

```
<html>
<head>
  <title>Registration Form</title>
  <style>
    body {
      font-family: Arial;
      background: #f2f2f2;
    }
    .form-box {
      width: 350px;
      margin: 50px auto;
      background: white;
      padding: 25px;
      border-radius: 8px;
```

```

    }
    input {
        width: 100%;
        padding: 10px;
        margin-top: 5px;
        margin-bottom: 5px;
    }
    .error {
        color: red;
        font-size: 13px;
        margin-bottom: 10px;
    }
    button {
        width: 100%;
        padding: 10px;
        background: green;
        color: white;
        border: none;
        cursor: pointer;
    }
</style>
</head>
<body>
<div class="form-box">
    <h2>Registration Form</h2>
    <form id="registerForm">
        <label>Email:</label>
        <input type="email" id="email"
            placeholder="Enter email" required>
        <div id="emailError" class="error"></div>
        <label>Phone Number:</label>
        <input type="tel" id="phone"
            placeholder="Enter 10 digit phone number"
            maxlength="10" required>
        <div id="phoneError" class="error"></div>
        <button type="submit">Register</button>
    </form>
</div>
<script>
    const form = document.getElementById("registerForm");
    const email = document.getElementById("email");
    const phone = document.getElementById("phone");
    const emailError = document.getElementById("emailError");
    const phoneError = document.getElementById("phoneError");
    function validateEmail() {
        let emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        if (!emailPattern.test(email.value)) {
            emailError.textContent = "Enter a valid email address";
            return false;
        }
        emailError.textContent = "";
        return true;
    }
    function validatePhone() {
        let phonePattern = /^[6-9][0-9]{9}$/;

        if (!phonePattern.test(phone.value)) {

```

```

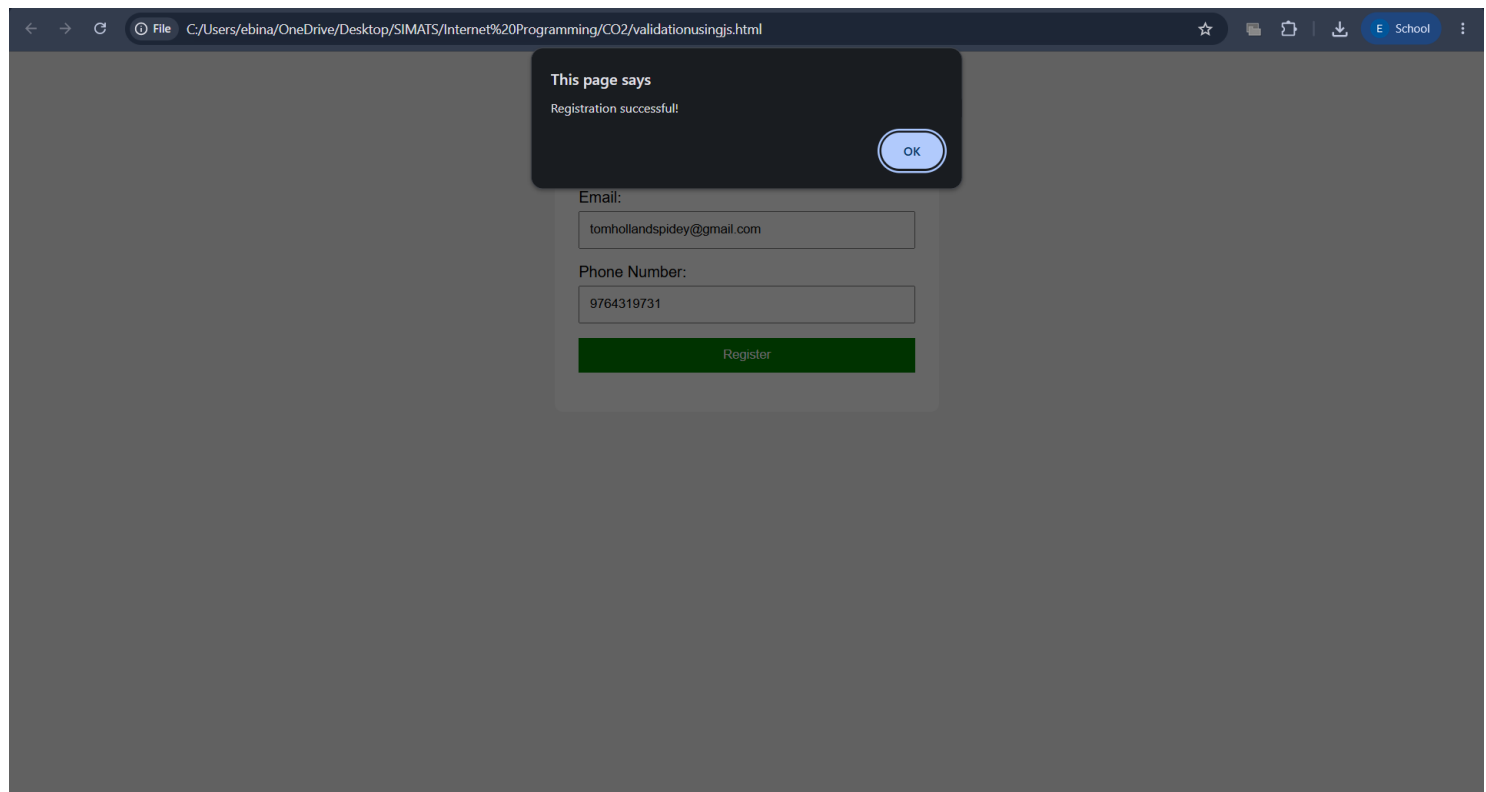
        phoneError.textContent =
            "Enter a valid 10-digit phone number";
        return false;
    }
    phoneError.textContent = "";
    return true;
}
email.addEventListener("input", validateEmail);
phone.addEventListener("input", validatePhone);

form.addEventListener("submit", function(event) {

    if (!validateEmail() || !validatePhone()) {
        event.preventDefault();
        alert("Please correct the errors.");
    } else {
        alert("Registration successful!");
    }
})
</script>
</body>
</html>

```

Output



Regular Expressions Used

Email:

`^[^s@]+@[^s@]+\.[^s@]+$`

It checks for:

- Username
- @ symbol
- Domain
- Dot in the domain

Phone:

`^[6-9][0-9]{9}$`

It accepts a **10-digit Indian mobile number beginning with 6, 7, 8, or 9.**

(d) Real-time validation

The following statements provide real-time validation:

```
email.addEventListener("input", validateEmail);
phone.addEventListener("input", validatePhone);
```

Therefore, error messages are displayed immediately while the user enters data.

(e) Evaluation

The validation system:

- Prevents invalid email addresses.
- Prevents invalid phone numbers.
- Provides immediate error messages.
- Prevents incorrect form submission.
- Improves user experience.

Further improvements

- Add password strength checking.
- Add password confirmation.
- Use server-side validation.
- Sanitize submitted data.
- Use HTTPS.
- Avoid exposing sensitive information in error messages.
- Add accessibility features such as aria-describedby.

Experiment 3: CSS Layout Debugging

Aim: To identify and correct CSS layout overlapping problems using the CSS Box Model, Flexbox, Grid, and positioning techniques.

(a) Causes of layout overlapping

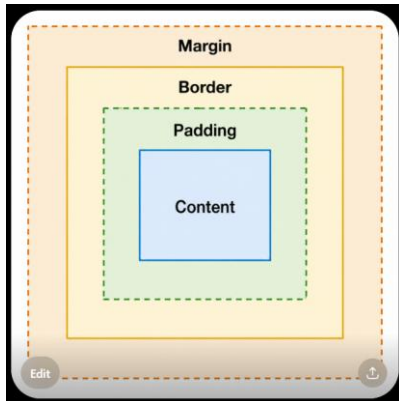
Common causes include:

1. Excessive use of position: absolute.
2. Fixed widths that do not adapt to screen size.
3. Negative margins.
4. Large padding and borders.

5. Incorrect z-index.
6. Fixed-height containers.
7. Lack of Flexbox or Grid.
8. Incorrect box-sizing.

(b) CSS Box Model

The CSS Box Model consists of:



It contains:

- **Content:** Actual text/image.
- **Padding:** Space between content and border.
- **Border:** Boundary around the element.
- **Margin:** Space outside the element.

Using:

`box-sizing: border-box;`

makes width and height calculations easier because padding and borders are included within the declared dimensions.

(c), (d) Debugged Layout Implementation

```
<html>
```

```
<head>
```

```
<title>CSS Layout Debugging</title>
```

```
<style>
```

```
* {
```

```
  box-sizing: border-box;
```

```
  margin: 0;
```

```
  padding: 0;
```

```
}

body {

    font-family: Arial;

}

header {

    background: #333;

    color: white;

    padding: 20px;

    text-align: center;

}

.container {

    display: flex;

    gap: 20px;

    max-width: 1000px;

    margin: 20px auto;

    padding: 20px;

}

.main {

    flex: 2;

    padding: 20px;

    border: 2px solid #ccc;

}

.sidebar {

    flex: 1;

    padding: 20px;

    border: 2px solid #ccc;

}

.cards {
```

```
display: grid;

grid-template-columns: repeat(2, 1fr);

gap: 15px;

margin-top: 20px;
}
```

```
.card {

padding: 20px;

border: 1px solid #999;

background: #f5f5f5;
}
```

```
footer {

background: #333;

color: white;

padding: 20px;

text-align: center;
}
```

```
/* Tablet and Mobile */
```

```
@media (max-width: 700px) {

.container {

flex-direction: column;
}

.cards {

grid-template-columns: 1fr;
}

.main,

.sidebar {

width: 100%;
}
```

```
    }

}

</style>

</head>

<body>

<header>

    <h1>CSS Layout Debugging</h1>

</header>

<div class="container">

    <main class="main">

        <h2>Main Content</h2>

        <div class="cards">

            <div class="card">

                <h3>Card 1</h3>

                <p>Product information.</p>

            </div>

            <div class="card">

                <h3>Card 2</h3>

                <p>Product information.</p>

            </div>

            <div class="card">

                <h3>Card 3</h3>

                <p>Product information.</p>

            </div>

            <div class="card">

                <h3>Card 4</h3>

                <p>Product information.</p>

            </div>

        </div>

    </main>

</div>
```

```
</div>

</div>

</main>

<aside class="sidebar">

  <h3>Sidebar</h3>

  <p>Additional information.</p>

</aside>

</div>

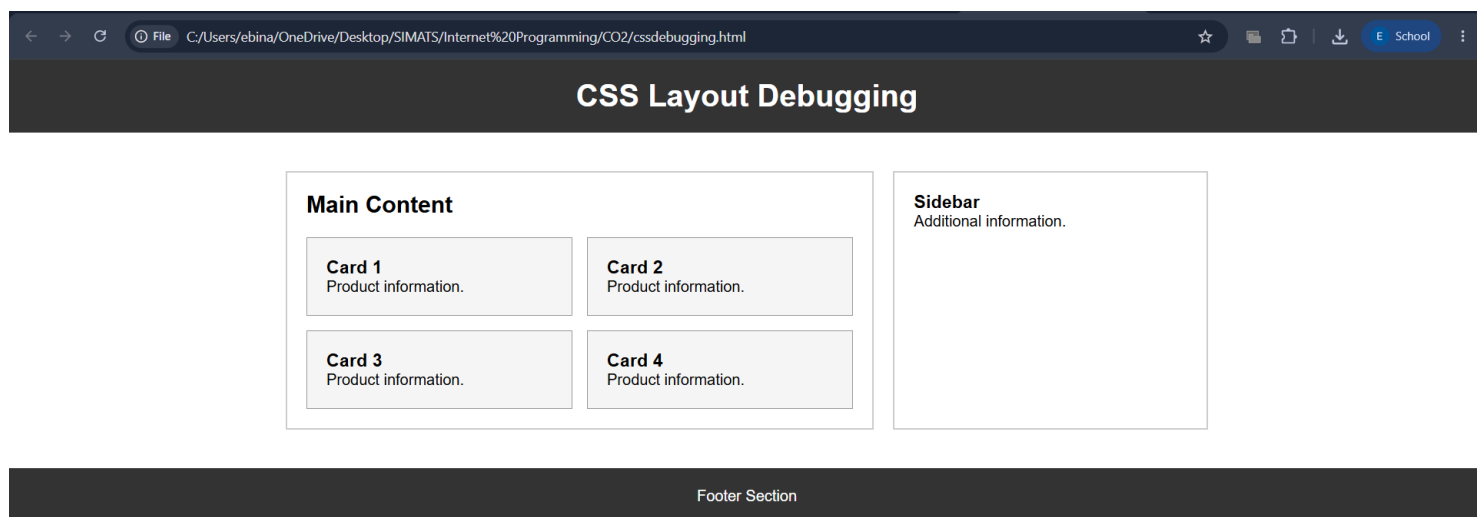
<footer>

  <p>Footer Section</p>

</footer>

</body>

</html>
```



Debugging techniques used

1. Flexbox

```
.container {  
  display: flex;  
  gap: 20px;  
}
```

Flexbox keeps the main content and sidebar properly aligned.

2. Responsive Flexbox

```
@media (max-width: 700px) {  
  .container {  
    flex-direction: column;  
  }  
}
```

The sidebar moves below the main content on small screens.

3. CSS Grid

```
.cards {  
  display: grid;  
  grid-template-columns: repeat(2, 1fr);  
}
```

Cards are automatically distributed into columns.

4. Preventing box-sizing problems

```
* {  
  box-sizing: border-box;  
}
```

This prevents padding and borders from unexpectedly increasing element width.

5. Avoiding unnecessary absolute positioning

Instead of:

```
position: absolute;
```

Flexbox and Grid are used for the main page structure.

(e) Evaluation

The corrected layout is more effective because:

- Content does not overlap when the browser is resized.
- Main content and sidebar remain properly aligned.
- Cards automatically adjust their size.
- Mobile users get a single-column layout.
- Tablet and desktop users get better use of available space.
- Flexbox and Grid reduce the need for manual positioning.

Best practices

1. Prefer **Flexbox and Grid** for page layouts.
2. Avoid unnecessary position: absolute.
3. Use box-sizing: border-box.
4. Use relative units where appropriate.
5. Use media queries for different screen sizes.
6. Avoid fixed heights for content containers.
7. Test layouts at multiple browser widths.
8. Use browser Developer Tools to inspect margins, padding, and element dimensions.